NLP Based User Intent Classification Using Machine Learning

COURSE PROJECT REPORT
Submitted by

Zayed Bin Hassan
ASI23CA064

Under the guidance of

Dr.SARIKA S

To
APJ Abdul Kalam Technological University
In partial fulfillment of the requirements for the award of the Degree of
Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence)

Department of Artificial Intelligence and Data Science

ADI SHANKARA INSTITUTE OF ENGINEERING AND TECHNOLOGY, KALADY

OCTOBER 2026

NLP Based User Intent Classification Using Machine Learning

COURSE PROJECT REPORT
Submitted by

Zayed Bin Hassan
ASI23CA064

Under the guidance of

Dr.Sarika S

To

APJ Abdul Kalam Technological University
In partial fulfillment of the requirements for the award of the Degree of
Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence)



Department of Artificial Intelligence and Data Science

ADI SHANKARA INSTITUTE OF ENGINEERING AND TECHNOLOGY, KALADY

OCTOBER 2026

ADI SHANKARA INSTITUTE OF ENGINEERING AND TECHNOLOGY, KALADY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE)



CERTIFICATE

Certified that this is a bonafide record of the course project entitled

NLP Based User Intent Classification Using Machine Learning

Submitted by

Zayed Bin Hassan
ASI23CA064

during the year 2026-27 in partial fulfillment of the requirement for the award of the degree of
Bachelor of Technology in Computer Science (Artificial Intelligence) and Engineering

Faculty in Charge

ACKNOWLEDGMENT

I express my sincere gratitude to Dr. Sarika S for her guidance and support during this course project. Her feedback helped refine the study of natural language processing and the presentation of the intent-classification system.

I thank the Department of Artificial Intelligence and Data Science, Adi Shankara Institute of Engineering and Technology, Kalady, for the academic environment supporting this work. I acknowledge Bitext for the customer-support dataset and the maintainers of NLTK, scikit-learn, React and the other open-source tools used in the project.

Zayed Bin Hassan
ASI23CA064

ABSTRACT

IntentLab is an NLP-based system that classifies short English customer-support messages into eight intents. NLTK tokenization preserves meaningful word structure and negation, while TF-IDF represents unigrams and bigrams. Logistic Regression and Multinomial Naive Bayes are compared using validation macro-F1, and the selected classifier serves predictions through a Python API and a React web interface. The application shows ranked predictions, signed feature evidence, preprocessing steps and an uncertainty outcome for messages with insufficient evidence.

The experiment uses 6,989 deduplicated messages from an eight-intent subset of Bitext's hybrid-synthetic customer-support dataset. Template and near-duplicate grouping precede a reproducible group-based split into 4,851 training, 1,083 validation and 1,055 test messages. A fresh reproduction on the saved partitions obtained test accuracy of 99.6209% and macro-F1 of 0.9940 for Multinomial Naive Bayes. Four test errors concern misspelled or concatenated cancellation requests.

IntentLab also supports ordered batch analysis, CSV export, browser-local history and measured model insights. Fresh verification passed 32 Python tests, seven frontend tests and 12 desktop/mobile browser tests, and the production frontend build completed. These checks establish local software behavior; the hybrid-synthetic dataset scores do not establish performance on real customer traffic. The report covers the methodology, implementation, measured results, screenshots, limitations and future scope.

TABLE OF CONTENTS

Sl. No.	Contents	Page No.
1	**Abstract**	5
2	**Chapter 1: Introduction**	7
2.1	Problem Statement	7
2.2	Objectives	7
2.3	Scope of the Project	7
3	**Chapter 2: Methodology / System Design**	8
3.1	Proposed Methodology	8
3.2	Tools and Technologies Used	9
3.3	System Architecture / Workflow	9
4	**Chapter 3: Implementation and Results**	10
4.1	Implementation Details	10
4.2	Results and Discussion	10
4.3	Testing and Screenshots	13
5	**Chapter 4: Conclusion and Future Scope**	18
	References	19

CHAPTER 1 INTRODUCTION

Customer-support systems receive short requests about orders, refunds, payments and account access. Identifying the user's intent can help organize these requests and suggest an appropriate workflow. IntentLab studies this task as supervised text classification and exposes the prediction process through an interactive interface.

1.1 Problem Statement

Develop a reproducible classifier that maps a short English customer-support message to one of eight supported intents and indicates when the available evidence is insufficient. The system should provide transparent model output, compare classical learning approaches and preserve clear separation between training, validation and held-out evaluation.

1.2 Objectives

The objectives are to apply NLTK tokenization meaningfully, preserve negation, represent words and phrases with TF-IDF, compare Logistic Regression and Multinomial Naive Bayes, select a model using validation macro-F1 and report held-out per-intent results. The interface should support single-message and batch analysis, signed feature evidence, preprocessing inspection and useful feedback for uncertain input.

1.3 Scope of the Project

The supported intents are Track order, Cancel order, Refund policy, Request refund, Payment issue, Account access, Get invoice and Contact support. Each message is treated as a single-intent English request. The system does not generate support replies, analyze sentiment or extract structured entities for downstream operations.

Training takes place offline; the API loads a saved classifier for inference. The project uses no paid AI API and requires no runtime NLTK corpus downloads. Uncertainty checks are heuristics around an in-domain classifier and are not a validated detector of every unsupported request. Local operation is verified; hosted Vercel deployment remains unverified.

CHAPTER 2 METHODOLOGY AND SYSTEM DESIGN

2.1 Proposed Methodology

The training workflow selects eight supported labels from the source dataset, removes repeated message/label pairs, forms template groups and creates separate training, validation and test partitions. Each candidate pipeline fits its TF-IDF representation only on training messages. Validation macro-F1 chooses the model; held-out test results are used for final reporting.



Figure 2.1 IntentLab training and inference workflow

2.2 Dataset and Partitioning

The source is Bitext's customer-support dataset, pinned to revision 72ea220, with the full revision recorded in provenance.json [1]. It is hybrid synthetic and is distributed under CDLA-Sharing-1.0. The project retains provenance, dataset checksums and the license notice. The full source has 27 intents; this project uses an eight-intent subset with 6,989 messages after deduplication.

Partition	Messages	Purpose
Training	4851	Fit vocabulary, IDF and model parameters
Validation	1083	Select model and probability threshold
Test	1055	Report final held-out performance

Entity/number-normalized templates and within-intent character-trigram cosine similarity of at least 0.92 form connected groups before an approximate 70/15/15 stratified group split with seed 42. The saved split contains 6,227 groups. Original seed identifiers are unavailable, so grouping cannot guarantee that every related synthetic template is separated.

2.3 Tools and Technologies Used

Technology	Purpose
Python, NumPy and SciPy	Training, inference and sparse feature operations
NLTK	Treebank word tokenization without corpus downloads
scikit-learn and joblib	TF-IDF, model fitting, metrics and persistence
React, TypeScript and Vite	Interactive frontend and production build
Python HTTP handler	Classification and model-readiness endpoints
pytest and Vitest	Backend and frontend unit tests
Playwright and Axe	Desktop/mobile workflows and accessibility checks

2.4 Preprocessing and Candidate Models

The tokenizer lowercases messages, replaces double-braced placeholders with entity, applies TreebankWordTokenizer and retains tokens containing alphabetic characters. Handling of cannot preserves a consistent negation token for features and explanations. The implementation does not apply stemming, lemmatization or general stopword removal to the model input. A stopword list is used separately when measuring content-vocabulary overlap.

Both candidates use TF-IDF word unigrams and bigrams, min_df=2 and sublinear_tf=True, with the shared tokenizer and lowercase=False because normalization occurs within tokenization. Logistic Regression uses C=4, max_iter=1000 and random_state=42. Multinomial Naive Bayes uses alpha=0.3. Validation macro-F1 selects Naive Bayes.

2.5 System Architecture and Workflow

intentlab/nlp.py contains tokenization and input validation; classifier.py loads the saved model and computes rankings, uncertainty and evidence. intentlab/http.py provides shared request handling, with api/classify.py and api/health.py exposing deployment entrypoints. The React frontend sends messages to the API and displays the actual returned results.

The API accepts one to 25 messages per batch, each at most 2,000 characters, and bounds the request body at 220 KB. Batch validation is atomic: invalid input fails before inference starts. Responses preserve input order and include status, label, score, top-three predictions, normalized text, tokens, signed evidence, version and server processing time.

2.6 Uncertainty and Explanation Design

The probability threshold of 0.40 was selected on in-domain validation data. Additional checks require a top-two score margin of 0.15, content-vocabulary overlap of 0.35 and three tokens. Insufficient evidence produces Needs clarification. These are heuristics; scores remain uncalibrated.

Naive Bayes evidence is TF-IDF multiplied by the target class feature log-likelihood minus the mean across other classes. Signed contributions describe the model calculation, rather than causal explanations or percentages.

CHAPTER 3 IMPLEMENTATION AND RESULTS

3.1 Implementation Details

IntentLab is implemented in the supplied local course-project workspace. Source modules, the saved model, dataset partitions, provenance, error records and the frontend model report are included. The project repository is https://github.com/ZayedBH/NLP_Intent_Classification. Hosted deployment remains unverified. Model reproduction and local application verification were performed on 15 October 2026.

Component	Implementation
scripts/train.py	Grouping, split construction, model comparison and export
models/intent-model.joblib	Trusted saved pipeline, threshold and model report
data/splits.json	Actual training, validation and test partitions
public/model-report.json	Measured results used by the insights interface
intentlab/ and api/	Shared preprocessing, inference and HTTP boundary
src/	Playground, batch analysis, insights and methodology views

Both candidate models were fitted again using the saved training partition and evaluated on the saved validation and test partitions. Their accuracy and macro-F1 values matched the recorded report. The selected retrained model also produced the same test predictions as the shipped saved model. Group construction was inspected in the source; this reproduction reused the saved partitions.

3.2 Results and Discussion

Model	Partition	Accuracy	Macro-F1
Logistic Regression	Validation	98.7996%	0.9832
Logistic Regression	Test	99.5261%	0.9933
Multinomial Naive Bayes	Validation	99.2613%	0.9905
Multinomial Naive Bayes	Test	99.6209%	0.9940

Naive Bayes was selected using validation macro-F1 of 0.9905. On the test partition it classified 1,051 of 1,055 messages correctly, giving accuracy of 99.6209% and macro-F1 of 0.9940. Logistic Regression achieved test accuracy of 99.5261% and macro-F1 of 0.9933. Macro-F1 gives each intent equal weight despite unequal class support.

3.2.1 Per Intent Test Results

Intent	Precision	Recall	F1	Support
Cancel order	1.0000	0.9394	0.9688	66
Refund policy	1.0000	1.0000	1.0000	150
Contact support	1.0000	1.0000	1.0000	149
Get invoice	1.0000	1.0000	1.0000	139
Request refund	1.0000	1.0000	1.0000	137
Payment issue	1.0000	1.0000	1.0000	148
Account access	1.0000	1.0000	1.0000	147
Track order	0.9675	1.0000	0.9835	119

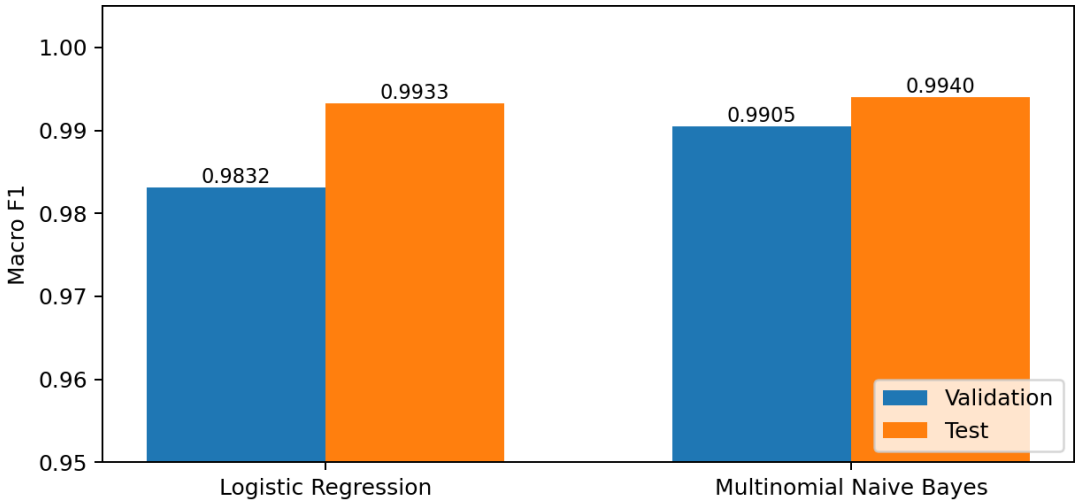


Figure 3.1 Reproduced validation and test macro-F1 comparison

Cancel order has the lowest recall, while Track order loses precision because it receives the four cancellation errors. The other six intents have no test errors in this partition. These results are specific to the hybrid-synthetic subset and do not establish equivalent performance on live support messages.

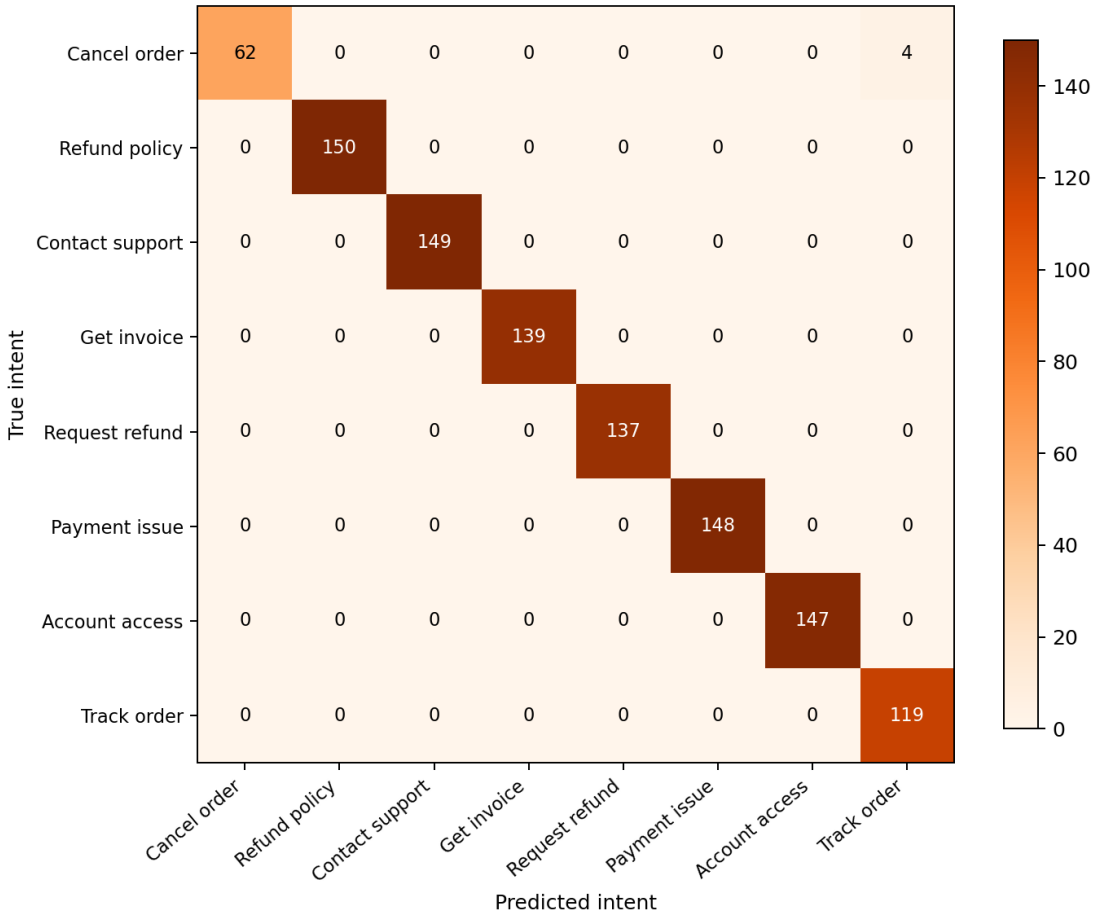


Figure 3.2 Test confusion matrix of the selected Naive Bayes model

3.2.2 Error Analysis

All four held-out mistakes are cancellation requests predicted as Track order. The recorded examples contain tocancel, cance, canmcel or cancelong. These misspellings or concatenated words weaken the cancellation features while retaining order-related vocabulary. They provide concrete evidence of limited robustness to surface-form variation.

The reported model comparison uses the classifiers' direct predictions. It does not report coverage or accepted accuracy after all application uncertainty heuristics. Those outcomes should be evaluated separately before treating Needs clarification as a validated open-set recognition mechanism.

INTENT PREDICTION AND FEATURE EVIDENCE

3.3 Testing and Screenshots

This fresh screenshot shows the running application classifying an order-tracking request. The ranked prediction and signed feature evidence come from the saved Naive Bayes pipeline. The interface lets the user inspect the model output instead of presenting a label alone.

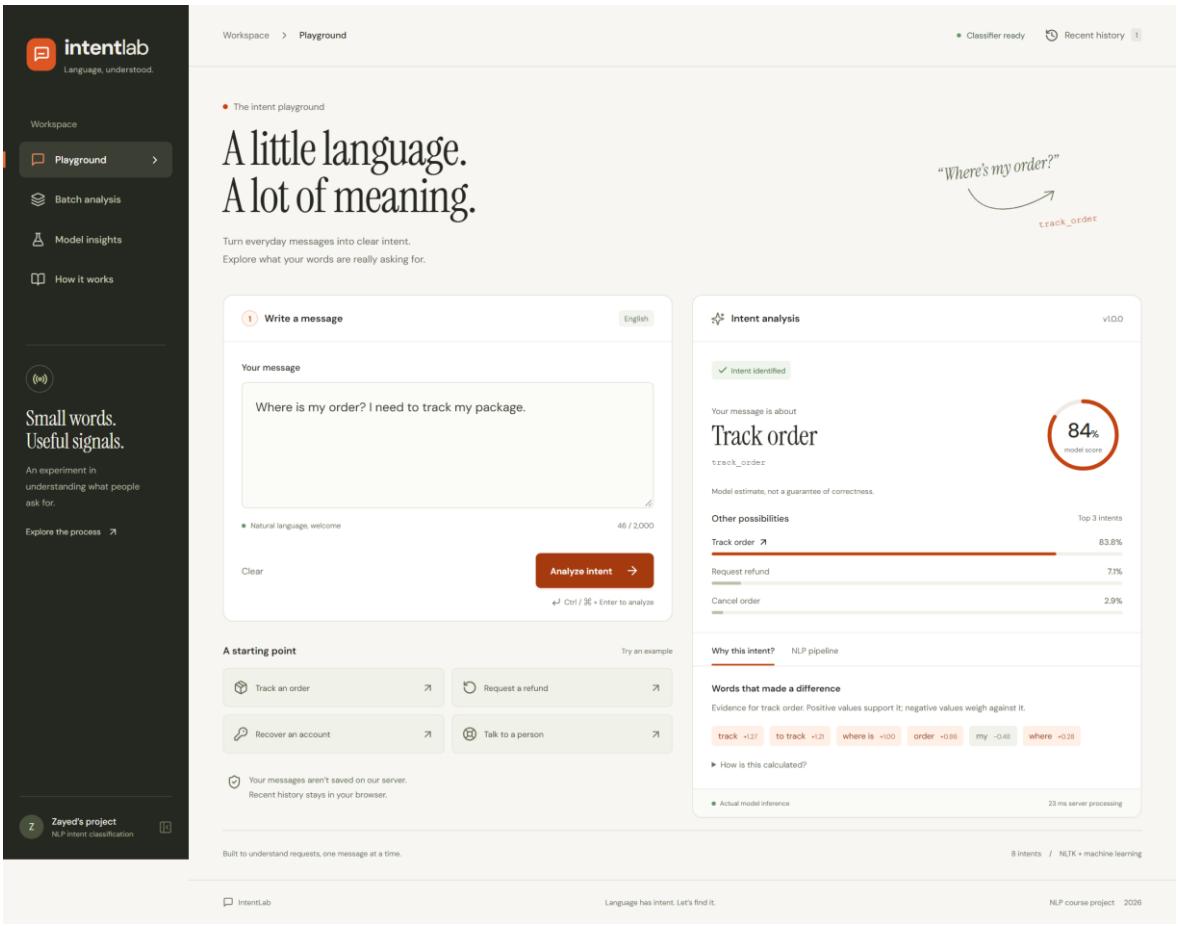


Figure 3.3 Fresh local screenshot of intent prediction and feature evidence

NLP PROCESSING VIEW

The NLP pipeline view exposes the normalized text and NLTK tokenization used by the actual classifier. It makes preprocessing visible for demonstration and discussion without implying that a generative model is interpreting the request.

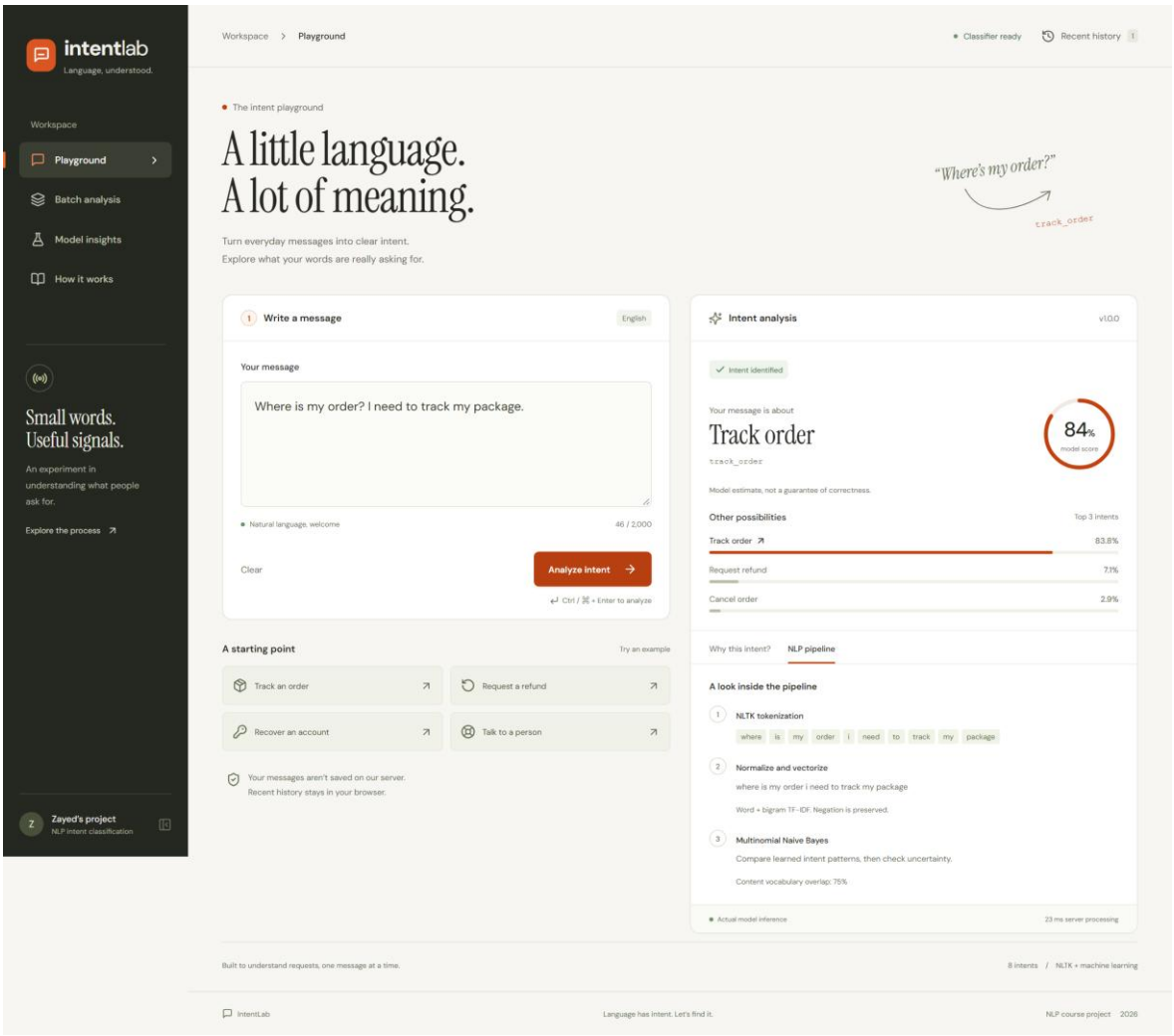


Figure 3.4 Fresh local screenshot of the NLP processing steps

UNCERTAINTY HANDLING

The unrelated request about black holes returned Needs clarification in the fresh browser check. Low overlap with the support vocabulary, short inputs or weak class separation can trigger this state. One successful example does not establish rejection accuracy for all unsupported requests.

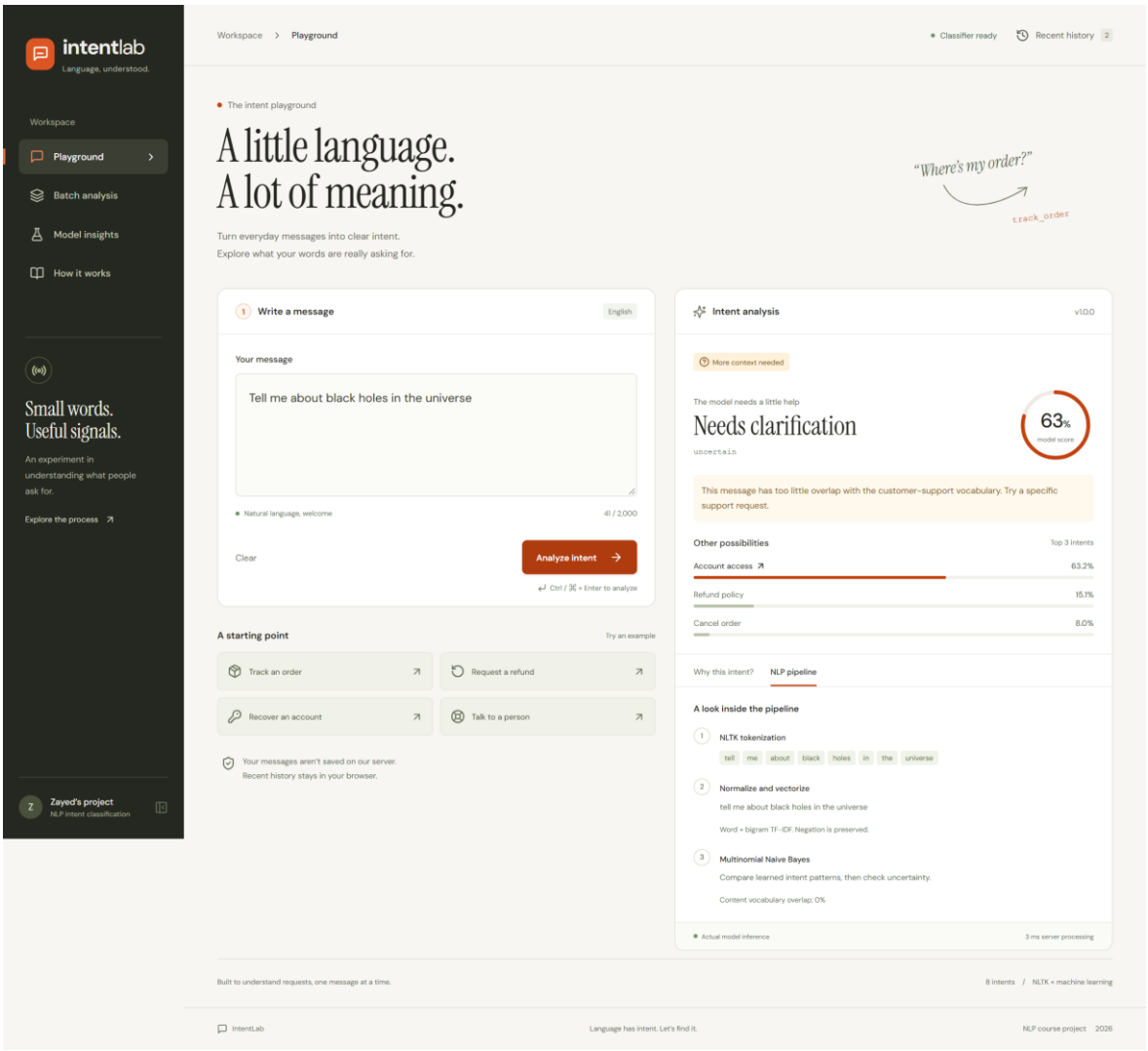


Figure 3.5 Fresh local screenshot of the uncertainty outcome

BATCH ANALYSIS

Batch analysis keeps each message paired with its prediction and supports CSV export. The browser check verified row order for cancellation and account-access examples. Pending-request controls prevent text edits from changing the meaning of a batch whose result is still being processed.

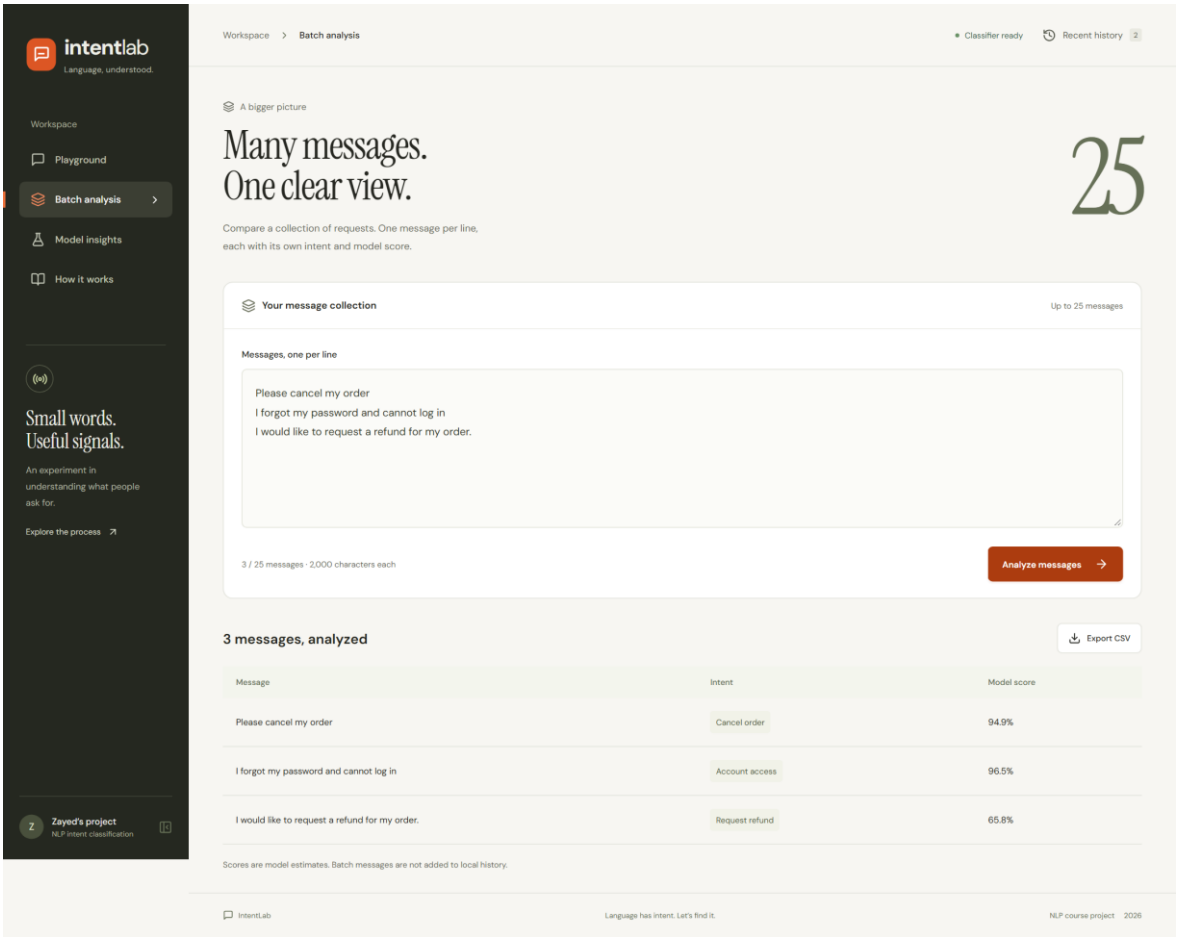


Figure 3.6 Fresh local screenshot of batch intent analysis

3.4 Software Verification

Check	Fresh result
Python suite	32 tests passed
Frontend suite	7 tests passed
Production build	TypeScript check and Vite build passed
Desktop/mobile browser suite	12 tests passed
End-to-end prediction	Real Python classification, evidence and token inspection passed
Uncertainty and validation	Blank input handling and unrelated-text example passed
Batch and export	Row pairing, pending-request behavior and CSV download passed
History	Persistence, recovery and clear-history behavior passed
Accessibility	Selected Axe WCAG A/AA rules passed in four initial workspace views
Model reproduction	Both candidates matched recorded accuracy and macro-F1 on saved partitions

The API does not persist message bodies to an application database or message log. The browser can retain up to ten recent analyses locally, including their text, so the history should be cleared on shared machines. CSV output applies spreadsheet-formula protection. Local tests and build results do not verify Vercel packaging, cold starts or hosted concurrency.

3.5 Interpretation and Evaluation Limits

The dataset is hybrid synthetic, related templates may remain across groups and original generation-seed IDs are unavailable. High scores can overestimate performance on real customer traffic. The eight labels exclude many support requests, and mixed-intent messages require a richer task definition. Uncertainty heuristics can still accept unsupported messages containing familiar vocabulary.

Automated accessibility checks apply to selected rules and states. Browser tests establish the exercised workflows and do not replace a complete manual accessibility audit. The test evidence is software verification, separate from the 1,055-message held-out model evaluation.

CHAPTER 4 CONCLUSION AND FUTURE SCOPE

4.1 Conclusion

IntentLab combines an auditable classical NLP classifier with an interactive analysis interface. The project provides grouped data partitions, training-only feature fitting, validation-based model selection, test metrics, error analysis, signed evidence and useful uncertainty feedback. Both candidate models reproduced their recorded scores, and the saved Naive Bayes model agreed with the selected retrained pipeline on all test predictions.

The measured test accuracy of 99.6209% describes this particular hybrid-synthetic dataset. The system's educational value is the complete path from text processing and feature representation to model comparison, explanation, API delivery and browser interaction. It does not establish unrestricted understanding of English or production customer-service reliability.

4.2 Limitations

The system handles English messages and one of eight intents at a time. Misspellings, concatenated words, domain shifts and multi-intent requests can cause errors. Probabilities are uncalibrated, and the uncertainty thresholds were not validated on a representative out-of-domain corpus. Template grouping reduces a known source of leakage but cannot prove the absence of every related synthetic example across partitions.

There is no login, production monitoring or server-side analytics in the current prototype. Browser-local history has privacy implications on shared devices. The prepared Vercel entrypoints require hosted deployment validation before any production availability claim.

4.3 Future Scope

Future work should add independently reviewed real support messages, temporal and cross-domain evaluation, calibrated scores and a labeled unsupported-intent test set. Character-level features or carefully evaluated normalization could improve robustness to the cancellation misspellings observed in the test errors.

Further extensions include multi-label prediction, multilingual support, additional intents and a contextual-model comparison using the same controlled partitions. Any integration with customer-service workflows should include human review and evaluation of both accepted errors and uncertainty coverage. Dataset sharing should retain the Bitext attribution and CDLA-Sharing-1.0 notices.

REFERENCES

[1] Bitext. Customer Support LLM Chatbot Training Dataset. https://github.com/bitext/customer-support-llm-chatbot-training-dataset. Pinned revision 72ea2203180d14a416b579c657033ecd21e59a11. Hybrid-synthetic data under CDLA-Sharing-1.0.

[2] Zayed Bin Hassan. NLP Intent Classification and IntentLab source code. https://github.com/ZayedBH/NLP_Intent_Classification. Local implementation inspected on 15 October 2026.

[3] IntentLab dataset and evaluation artifacts: data/provenance.json, data/splits.json, data/test-errors.json and public/model-report.json. Candidate-model results reproduced on saved partitions on 15 October 2026.

[4] Bird, S., Klein, E. and Loper, E. (2009). Natural Language Processing with Python. O'Reilly Media.

[5] Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.